

Análisis y Diseño de Software

Práctica 4: Herencia, Interfaces y Excepciones

Memoria

Autores: Claudia Saiz Escribano y Duna Puente Romera

Grupo de laboratorio: 2201

Fecha: 13 de abril de 2026

Tabla de contenidos

Decisiones y problemas resueltos	2
Problemas pendientes de resolver.....	4
Diagrama de clases.....	4
Pruebas realizadas.....	5

Decisiones y problemas resueltos

Apartado 1

Nos dimos cuenta de que con 5 atributos que pueden especificarse o ponerse por defecto al crear un nuevo sensor, si quisiéramos cubrir todas las posibilidades tendríamos que hacer 32 métodos para añadir un sensor. Para resolver esto redujimos el número a 7: uno de ellos en el cual no se especifican ninguno, uno en el cual se especifican todos, y los otros individuales en los que se especifica uno de los atributos únicamente.

Dentro de los sensores, establecimos una clase padre abstracta con los métodos y atributos comunes, y una subclase para cada tipo de sensor donde se especifican parámetros propios de cada especie de sensor. Es importante mencionar que, para diferenciar entre las distintas unidades de medida del sensor de temperatura, se ha creado una enumeración especificando la unidad, y distintos métodos privados para hacer la conversión a Celsius.

Luego, en el paquete de la estación meteorológica se creó una clase para especificar la ubicación geográfica, y dentro de la clase de la estación, los sensores se han instanciado como un *hash map* donde cada uno está identificado por su ID, lo que simplificaba el desarrollo de los métodos asociados.

En el caso de añadir un nuevo sensor, se creó una nueva excepción cuyo mensaje incluye el sensor que se pretendía añadir y el sensor preexistente con cuyo ID coincide. Por otro lado, a la hora de filtrar los sensores por tipo, se decidió crear una enumeración del tipo de sensor que lleve asociada el prefijo de los identificadores, lo que facilita enormemente la búsqueda.

Para acabar, se ha creado un atributo *Timer* encargado de realizar de manera simultánea las lecturas de los sensores. En este método también se ha instanciado un valor *Atomic Integer*, que permite llevar la cuenta de las lecturas máximas para no sobrepasar el valor especificado.

Apartado 2

Decidimos que la estrategia por defecto sería una estrategia aleatoria en la que el rango sería el mismo que el rango de medición del sensor, con una probabilidad nula de que el valor generado se encuentre fuera del rango.

Además, para evitar que en las estrategias cercana y media se pueda estancar el valor generado cuando el último o la media respectivamente son cero, se ha incorporado el atributo estático *valorMínimo*, que asegura que siempre habrá una variación mínima.

Apartado 3

Para este apartado, decidimos crear únicamente 3 tipos de conversores de temperatura, de manera que, por composición de ellos, se puede pasar desde cualquier unidad a otra. Con los conversores de presión, al medirse únicamente en hPa, solo se crearon dos conversores que permiten pasar a Pa y a mBar, pero no entre ellos ya que no sería necesario. Los sensores de

humedad solo miden en una unidad y no se puede pasar a otra, por lo que solo aceptan conversores de identidad.

Al añadir un nuevo sensor, se hacen las comprobaciones de que el tipo de conversor asociado sea de el mismo tipo (en el caso de conversor compuesto se checkean ambos), y que la unidad de entrada del conversor sea la misma que la unidad de medición del sensor.

En el caso de los conversores compuestos también se comprueba al crear uno que la unidad de salida del primero concuerde con la unidad de entrada del segundo. Se ha tomado la decisión de que no sea posible crear un conversor compuesto de tipo presión, ya que con los conversores existentes nunca concordarían las unidades, y es posible convertir los valores a todas las unidades de presión sin utilizarlo.

Apartado 4

A la hora de gestionar las alertas, se crearon 4 excepciones nuevas que pasaron a ser lanzadas por sus métodos correspondientes dentro de la clase Sensor.

Desde la estación meteorológica, las alertas se instanciaron con un *hash map* que relaciona cada sensor con su lista de excepciones. Asimismo, se añadió un nuevo método que filtra los sensores registrados para obtener solo aquellos válidos para realizar las lecturas. También se crearon verificaciones de manera que un mismo tipo de alerta solo se añada una vez en cada sensor.

Por último, se han creado todos los nuevos métodos relacionados con la calibración de los sensores y la eliminación de sus alertas asociadas.

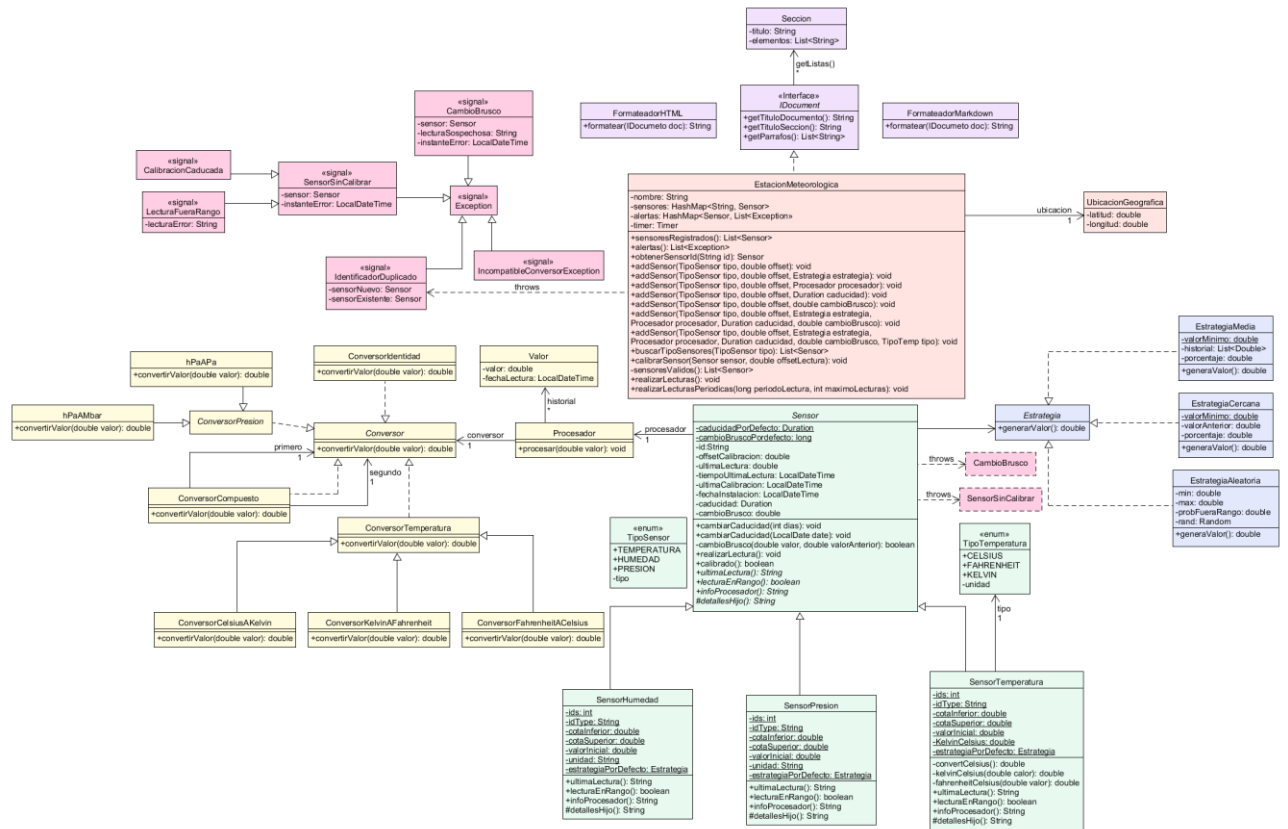
Apartado 5

En este apartado simplemente hemos implementado la interfaz *IDocument* en la clase de la estación meteorológica, de manera que se pudieran obtener los diferentes títulos, párrafos y secciones. Luego los diferentes formateadores se encargan de devolver una cadena con la información en el formato correspondiente.

Problemas pendientes de resolver

En esta práctica no tenemos ningún problema pendiente de resolver a futuro.

Diagrama de clases



Dentro del diagrama de clases hemos diferenciado los distintos paquetes por colores y, para mantener la claridad del diseño, hemos realizado una indicación en la clase Sensor sobre el tipo de excepciones que lanza.

Pruebas realizadas

Para la comprobación del correcto funcionamiento de esta práctica, se han seguido las indicaciones del enunciado y se ha realizado un programa *main* de prueba para cada apartado. En cada una de ellas se han comprobado tanto que se que hagan las comprobaciones necesarias en caso de que los parámetros de entrada o las condiciones no sean las correctas, como que el funcionamiento sea adecuado en casos normales.

Dentro de la prueba del apartado 1, probamos a añadir distintos sensores a la estación meteorológica con los diversos métodos posibles; luego comprobamos la funcionalidad del filtrado de los sensores según el tipo, buscamos un sensor en base a su ID y acabamos con una lectura puntual y otra periódica. Finalmente, se imprime la información de la estación.

En el caso de la prueba del apartado 2, empezamos creando un sensor que, por defecto, tiene asociada una estrategia aleatoria y comprobamos que, en cualquier caso, el valor generado se encuentra dentro del rango del tipo de sensor. A continuación, probamos a crear dos nuevas estrategias aleatorias: una que tiene porcentaje cero de salirse del rango y otra con porcentaje del ciento por ciento, y verificamos que los valores se generan según los respectivos criterios. Después, revisamos el correcto funcionamiento de las estrategias cercana y media, y en el caso de la última, que la generación de nuevos valores cambia la media.

El apartado 3 se encarga de probar las excepciones por conversor no válido y de verificar que en los constructores sin especificar el conversor por defecto es el Conversor Identidad. Finalmente, se prueba que añadiendo sensores con conversores específicos las lecturas se realizan correctamente.

El funcionamiento del cuarto test comienza añadiendo diversos sensores con una estrategia aleatoria por defecto y uno con una estrategia cercana. Los primeros tienen una alta probabilidad de generar una alerta por cambio brusco en las lecturas, pero por si diera el caso en que este estado no surge en ningún momento, se crea un sensor que garantiza que esta excepción no surgirá a menos que se fuerce, lo cual se realiza más adelante.

Se realiza una primera lectura de los sensores, luego calibramos uno de ellos para forzar una lectura fuera de rango, realizamos una segunda lectura y forzamos que la calibración de otro sensor distinto caduque. Acto seguido, hacemos una nueva lectura para que las alertas se actualicen y forzamos el cambio brusco del sensor mencionado al principio para ver que surge el *warning* asociado. Acabamos con una última lectura e imprimimos la información de la estación meteorológica para verificar que todo funciona correctamente.

Para acabar, la prueba del apartado 5 consiste en establecer una estación meteorológica, con sus respectivos sensores, realizar lecturas, imprimir tanto el formato HTML como el formato Markdown y comprobar que ambos se construyen como solicita el enunciado.